

RESUMO COMPLETO - COMPUTAÇÃO GRÁFICA

Para o Teste - 19 de Maio de 2026

ÍNDICE

1. Espaços de Coordenadas
 2. Iluminação
 3. Transformações Geométricas
 4. Curvas e Superfícies
 5. Texturas
 6. Culling e Partição
 7. Fichas de Consolidação
-

ESPAÇOS DE COORDENADAS

Espaço Local (Object Space / Modelling Space)

- **Definição:** Sistema de coordenadas relativo a um objecto (ou grupo de objectos)
- **Propósito:** Permite definir coordenadas relativas ao objecto
- **Origem:** No próprio objecto

Espaço Global (World Space)

- **Definição:** Engloba todo o universo de coordenadas absolutas
- **Propósito:** Expressar coordenadas de forma absoluta no mundo
- **Operações aqui:** Movimentos, detecção de colisões, iluminação

Espaço da Câmara (Camera Space / View Space)

- **Definição:** Sistema de coordenadas associado ao observador (câmara)
- **Origem:** Posição da câmara
- **Eixos:** Determinados pela orientação da câmara
- **OpenGL padrão:**
 - Câmara em origem (0, 0, 0)
 - Aponta na direcção -Z
 - Eixo Y para cima
 - Eixo X para a direita

Espaço do Ecrã (Screen Space)

- **Definição:** Espaço 2D onde é visualizado o mundo virtual
- **Resultado:** Projeção do espaço da câmara
- **Origem:** Centro do ecrã
- **Coordenadas:** Homogêneas (não são pixéis!)
- **Viewport:** Mapeamento entre espaço câmara e ecrã

Viewport

- **Zona de display:** Onde pretendemos desenhar
- **Função:** Funciona como janela que limita o espaço visível
- **Definições:**
 - Lados do View Frustum
 - Distância entre câmara e plano de projeção
 - Retângulo no plano de projeção
- **Ângulo:** Define o ângulo entre os lados do retângulo (FOV - Field of View)

Projeções

Projeção em Perspectiva

- **Características:** Mais realista
- **Propriedade:** Objetos ficam mais pequenos quanto mais longe
- **Fórmula:** Divisão pela coordenada Z (profundidade)
- **Uso:** Games, visualizações 3D

Projeção Paralela (Ortográfica)

- **Características:** Linhas paralelas mantêm-se paralelas
- **Propriedade:** Sem distorção de profundidade
- **Fórmula:** Projeção sem divisão por Z
- **Uso:** CAD, plantas, desenho técnico

ILUMINAÇÃO

Fundamentos de Iluminação

Lei do Coseno

A intensidade de luz refletida por uma superfície depende de: 1. **Intensidade da luz** (I) 2. **Orientação do objeto** em relação à fonte (ângulo θ) 3. **Distância à fonte de luz** (d)

Equação Básica

$$I(d) = I_0 \times \cos(\theta) \times \frac{1}{d^2}$$

Onde: - **I** = intensidade do emissor - θ = ângulo de incidência entre normal e raio de luz - **d** = distância à fonte - Considera-se apenas **cos() > 0**

Componente Difusa (Reflexão Lambertiana)

A intensidade refletida é **uniforme em todas as direções**:

$$I_d = K_d \times I_l \times \max(0, \vec{n} \cdot \vec{l})$$

Variáveis: - **K_d**: Coeficiente de reflexão difusa [0, 1] - **I_l**: Intensidade da fonte de luz - $\vec{n}$: Vetor normal da superfície (normalizado) - $\vec{l}$: Vetor direção da luz (normalizado) - $\vec{n} \cdot \vec{l}$: Produto interno (substitui cos quando vetores normalizados)

Componente Ambiente

Simula de forma básica as interações entre objetos:

$$I_a = K_a \times I_{ga}$$

Variáveis: - **K_a**: Coeficiente de reflexão ambiente [0, 1] - **I_{ga}**: Intensidade da luz ambiente global (típicamente 0.2, 0.2, 0.2) - **Afeta**: Todos os pontos igualmente - **Limitação**: Não considera relação espacial objeto-luz

Atenuação pela Distância

Fórmula Física (Incorreta em CG)

$$f_{att} = \frac{1}{d^2}$$

Problema: - Grandes variações para luz próxima - Intensidade muito pequena para luz distante

Fórmula de Compromisso (Usada em OpenGL)

$$f_{att} = \min \left(\frac{1}{c_1 + c_2 \cdot d + c_3 \cdot d^2}, 1.0 \right)$$

Parâmetros: - **c**: Constante que evita denominador muito pequeno - **c**: Coeficiente linear - **c**: Coeficiente quadrático - **Garantia:** $f_{att} \leq 1.0$

Componente Especular

Modelo Phong

Para superfícies brilhantes, há uma mancha clara cuja posição depende do **observador**:

$$I_s = K_s \times I_l \times \max(0, \cos(\alpha))^n$$

Variáveis: - **K_s**: Coeficiente de especularidade [0, 1] - **I_l**: Intensidade da luz - α : Ângulo entre vetor refletido e vetor da câmara - **n**: Coeficiente de brilho (shininess) [0, 128]

Modelo Blinn-Phong (Alternativa)

Usa o **meio-vetor** em vez de vetores refletidos:

$$I_s = K_s \times I_l \times \max(0, \vec{n} \cdot \vec{h})^n$$

Vantagens: - Computacionalmente mais eficiente - Melhor comportamento em alguns casos - $\vec{h}$ = meio-vetor = $\frac{\vec{l} + \vec{v}}{|\vec{l} + \vec{v}|}$

Interpretação do Shininess (n)

- **n = 0.1**: Mancha grande e suave (material baço)
- **n = 8**: Mancha média (plástico brilhante)
- **n = 32**: Mancha pequena (metal brilhante)
- **n = 128**: Mancha muito pequena (metal espelho)

Equação Completa de Iluminação

$$I_{total} = K_e + (K_a \times I_{ga}) + f_{att} \times [(K_d \times I_l \times \max(0, \vec{n} \cdot \vec{l})) + (K_s \times I_l \times \max(0, \vec{n} \cdot \vec{h})^n)]$$

Componentes: - **K_e**: Cor emissiva (luz emitida pelo material) - **Ambiente**: Luzes indiretas simuladas - **Difusa**: Reflexão base do material - **Especular**: Reflexos brilhantes - **f_{att}**: Atenuação pela distância

Materiais em OpenGL

Componentes de Cor RGBA

Cada componente tem valores em [0.0, 1.0]:

```
// Exemplo: Material metálico vermelho
float mat_dif[] = {0.8, 0.2, 0.2, 1.0}; // Difusa (vermelho brilhante)
float mat_amb[] = {0.2, 0.05, 0.05, 1.0}; // Ambiente (vermelho escuro)
float mat_spe[] = {1.0, 1.0, 1.0, 1.0}; // Especular (branco para metal)
```

```
float mat_emi[] = {0.0, 0.0, 0.0, 1.0};    // Emissão (não emite luz)

glMaterialfv(GL_FRONT, GL_DIFFUSE, mat_dif);
glMaterialfv(GL_FRONT, GL_AMBIENT, mat_amb);
glMaterialfv(GL_FRONT, GL_SPECULAR, mat_spe);
glMaterialfv(GL_FRONT, GL_EMISSION, mat_emi);
glMaterialf(GL_FRONT, GL_SHININESS, 128);
```

Componentes de Material

Componente	Descrição	Intervalo
GL_DIFFUSE	Reflexão base	[0, 1] RGBA
GL_AMBIENT	Luz ambiente	[0, 1] RGBA
GL_SPECULAR	Reflexos brilhantes	[0, 1] RGBA
GL_EMISSION	Luz própria	[0, 1] RGBA
GL_SHININESS	Brilho	[0, 128]

Targets de Aplicação

- **GL_FRONT**: Faces frontais do polígono
- **GL_BACK**: Faces traseiras
- **GL_FRONT_AND_BACK**: Ambas as faces
- **GL_AMBIENT_AND_DIFFUSE**: Aplica a ambas

Luz Ambiente Global Padrão

```
(0.2, 0.2, 0.2, 1.0) // Cinzento escuro para evitar preto total
```

Modelos de Shading

1. Flat Shading

Processo: 1. Uma **normal por triângulo** (normal do polígono) 2. Cor calculada para **um único ponto** do triângulo 3. **Todo o triângulo** pintado com essa cor

Características: - Aspecto **facetado muito pronunciado** - Requisitos: Luz infinitamente distante, câmara infinitamente distante - Problema: **Bandas de Mach** (disparidade entre diferença real e percebida)

2. Gouraud Shading (Interpolação de Cores)

Processo: 1. Calcular **normal por vértice** = média das normais da superfície original 2. Calcular **cor em cada vértice** usando equação de iluminação completa 3. **Interpolar cores** linearmente entre vértices para pontos interiores

Vantagens: - Elimina aspecto facetado - Luz não precisa estar infinitamente distante

Limitações: - Superfície continua facetada nas arestas - Manchas especulares **não reproduzidas fielmente** - **Problema Clássico:** Luz circular que inclui só um canto $\rightarrow$ triângulo todo aparece não iluminado

3. Phong Shading (Interpolação de Normais)

Processo: 1. Calcular **normal por vértice** = normal da superfície original nos vértices 2. **Interpolar normais** linearmente entre vértices para pontos interiores 3. Calcular **cor em cada ponto** usando a **normal interpolada** na equação completa

Vantagens: - Resolve problemas do Gouraud - Manchas especulares corretas - Aspecto mais suave e realista

Desvantagem: Computacionalmente intensivo (antes problema, agora trivial com shaders)

Comparação Visual

Aspecto	Flat	Gouraud	Phong
Aspecto	Facetado	Suave	Muito Suave
Manchas Especulares	Incorretas	Interpoladas (errado)	Corretas
Iluminação	Por triângulo	Por vértice + interpolação	Por pixel
Custo Computacional	Muito baixo	Baixo	Moderado
Banda de Mach	Visível	Menos visível	Invisível

TRANSFORMAÇÕES GEOMÉTRICAS

Matrizes Homogêneas 4×4

Por que 4×4 ?

- **3D convencional:** Matrizes 3×3 só fazem rotações e escalas
- **Translação:** Não é representável em 3×3
- **Solução:** Coordenadas homogêneas (x, y, z, w)
- **Notação 4×4 :** Permite translação + rotação + escala numa única matriz

Translação

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Aplicação:

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Escala

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotação em Torno de X

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotação em Torno de Y

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotação em Torno de Z

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Composição de Transformações

Ordem Importante!

- **Rotação depois Translação:** Objecto roda e depois move (eixo global)
- **Translação depois Rotação:** Objecto move e depois roda em torno da origem
- **Em código:** Aplicar transformações na **ordem inversa**

```
// Rotacionar 45° em Z, depois transladar (5, 0, 0)
// Código: T * R (porque matriz se aplica da direita)
glTranslatef(5, 0, 0); // Escreve segundo
glRotatef(45, 0, 0, 1); // Escreve primeiro (será aplicado primeiro)
```

Câmaras

Câmara em OpenGL

Configuração padrão: - Posição: (0, 0, 0) - Olha para: (0, 0, -1) - Eixo Y para cima: (0, 1, 0)

gluLookAt

```
gluLookAt(  
    eye_x, eye_y, eye_z,      // Posição da câmara  
    center_x, center_y, center_z, // Para onde olha  
    up_x, up_y, up_z         // Vetor "para cima"  
);
```

Exemplo:

```
// Câmara em (5, 5, 5), olhando para origem (0, 0, 0)  
gluLookAt(5, 5, 5,      // Eye  
          0, 0, 0,      // Center  
          0, 1, 0);    // Up vector
```

Coordenadas Esféricas

Movimento de câmara em torno de um ponto central:

Convenção UMinho (Oficial dos Slides):

$$x = r \times \cos(\alpha) \times \cos(\beta)$$

$$y = r \times \sin(\alpha)$$

$$z = r \times \cos(\alpha) \times \sin(\beta)$$

Variáveis: - **r**: Distância ao centro - (**alpha**): Ângulo vertical (elevação) a partir do plano horizontal XZ $[-\pi/2, \pi/2]$ - **Propriedade Crítica:** O Y é diretamente $r \sin(\alpha)$ (seno!) - Valor positivo = acima do plano XZ - Valor negativo = abaixo do plano XZ - (**beta**): Ângulo horizontal (azimute) em torno do eixo Y $[0, 2\pi]$ - 0° aponta na direção +X - 90° aponta na direção +Z

Nota: Esta é a convenção usada nos slides da UMinho. A variação com as clássicas coloca Y vertical ao invés.

Câmara Explorador (Primeira Pessoa)

Movimento: - **W/S**: Move para frente/trás - **A/D**: Move para esquerda/direita - **Mouse**: Roda a câmara (pitch, yaw)

Implementação:


```
// Vetor direção (frente)
front = (cos(yaw) * cos(pitch), sin(pitch), sin(yaw) * cos(pitch))

// Translação
position += speed * front // Para frente
position += speed * right // Para direita
```

Câmara FPS

Câmara de primeira pessoa com: - **Movimento:** WASD - **Controlo:** Mouse
look (yaw/pitch) - **Restrição:** Pitch limitado a $[-89^\circ, 89^\circ]$ (não virar cabeça para trás)

CURVAS E SUPERFÍCIES

Curvas Bezier

Bezier de Grau 1 (Reta)

$$B(t) = (1 - t)P_0 + tP_1, \quad t \in [0, 1]$$

Bezier de Grau 2 (Parábola)

$$B(t) = (1 - t)^2P_0 + 2(1 - t)tP_1 + t^2P_2$$

Bezier de Grau 3 (Cúbica)

$$B(t) = (1 - t)^3P_0 + 3(1 - t)^2tP_1 + 3(1 - t)t^2P_2 + t^3P_3$$

Forma Geral Bezier (Grau n)

$$B(t) = \sum_{i=0}^n B_{i,n}(t)P_i$$

Onde $B_{i,n}(t) = \binom{n}{i}(1 - t)^{n-i}t^i$ (Polinômios Bernstein)

Propriedades Bezier

- **Passa pelos extremos:** $B(0) = P_0$, $B(1) = P_n$
- **Tangente em P_0 :** Direção $P_1 - P_0$
- **Tangente em P_n :** Direção $P_n - P_{n-1}$
- **Convex Hull Property:** Curva está dentro do polígono de controlo
- **Suavidade:** Curva suave, sem picos

Algoritmo de De Casteljau

Processo recursivo para avaliar um ponto na curva Bezier:

Para Bezier cúbica com parâmetro t :

Nível 1:

$$Q_0 = (1 - t)P_0 + tP_1$$

$$Q_1 = (1 - t)P_1 + tP_2$$

$$Q_2 = (1 - t)P_2 + tP_3$$

Nível 2:

$$R_0 = (1 - t)Q_0 + tQ_1$$

$$R_1 = (1 - t)Q_1 + tQ_2$$

Nível 3:

$$S = (1 - t)R_0 + tR_1$$

S é o ponto $B(t)$ na curva!

Vantagem: Numericamente estável, usado em muitos sistemas

Curvas Hermite Cúbicas

Definição por: - **P** : Ponto inicial - **P** : Ponto final - **T** : Tangente em P - **T** : Tangente em P

$$B(t) = H_0(t)P_0 + H_1(t)T_0 + H_2(t)P_1 + H_3(t)T_1$$

Funções base Hermite:

$$H_0(t) = 2t^3 - 3t^2 + 1$$

$$H_1(t) = t^3 - 2t^2 + t$$

$$H_2(t) = -2t^3 + 3t^2$$

$$H_3(t) = t^3 - t^2$$

Curvas Catmull-Rom

Interpolação automática de pontos de controlo:

$$B(t) = \frac{1}{2} \begin{pmatrix} 1 & t & t^2 & t^3 \end{pmatrix} \begin{pmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{pmatrix} \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix}$$

Propriedades: - Passa pelos pontos de controlo (excepto extremos) - Usa 4 pontos, calcula curva entre P_0 e P_3 - Garantida suavidade C^1 (derivada contínua)
- Usada em **animação de câmaras e caminhos**

Animação com Catmull-Rom

Para cada frame:

```
tempo = frame / frames_totais [0, 1]

// Posição
posicao = Catmull_Rom(pontos_posicao, tempo)

// Tangente (velocidade)
tangente = derivada_Catmull_Rom(pontos_posicao, tempo)

// Rotação (olha na direção do movimento)
up_vector = (0, 1, 0) ou outro eixo
frente = normalize(tangente)
direita = cross(up, frente)
up_novo = cross(frente, direita)
```

Superfícies Bezier (Bezier Patches)

Bezier Quadrilateral (4×4 pontos de controlo)

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_{i,3}(u) B_{j,3}(v) P_{i,j}$$

Propriedades: - **Quadrilátero de 4 pontos de controlo:** Define uma curva Bezier em u - Para cada **curva em u** , é uma curva Bezier em v - **Continuidade:** C^0 com patches vizinhos (apenas posição) - Para continuidade C^1 : Arestas devem ter tangentes iguais

Cálculo de Normais

Para iluminação correta, necessário calcular normal em cada ponto:

$$\vec{n}(u, v) = \frac{\partial S}{\partial u}(u, v) \times \frac{\partial S}{\partial v}(u, v)$$

Processo: 1. Calcular derivada parcial em u 2. Calcular derivada parcial em v 3. Produto vetorial = normal 4. Normalizar

Classificação de Curvas Cúbicas

Todas estas curvas são cúbicas (grau 3): - **Bezier**: Controlo intuitivo, não passa pontos intermédios - **Hermite**: Controlo por extremos e tangentes - **Catmull-Rom**: Passa por todos os pontos

Diferenças na matriz de base M (primeira linha): - **Bezier**: [1, 0, 0, 0] (apenas P) - **Hermite**: [1, 0, 0, 0] (similar) - **Catmull-Rom**: [0, 1, 0, 0] (passa por P)

TEXTURAS

Definição e Aplicação

Texturing: Mapear imagens 1D, 2D ou 3D a primitivas geométricas

Tipos de Texturas

Tipo	Descrição	Uso
1D	Linha de pixéis	Gradientes, cores
2D	Imagem regular	Superfícies planas/polígonos
3D	Volume/Solid texture	Efeitos naturais, sólidos

Aplicações

1. **Simular materiais**: Madeira, granito, tijolos
2. **Substituir geometria**: Em vez de polígonos complexos
3. **Efeitos especiais**: Reflexos, refrações, lens flares
4. **Ambiente mapping**: Céu, reflexões

Sistema de Coordenadas de Textura

Espaço de textura: Dimensões (u, v) ou (s, t) - **u/s**: Coordenada horizontal [0, 1] - **v/t**: Coordenada vertical [0, 1] - **(0, 0)**: Canto inferior-esquerdo - **(1, 1)**: Canto superior-direito

3D adiciona w: Profundidade para texturas volumétricas

Aplicação de Texturas em OpenGL

Passo 1: Carregar e Criar Textura

```
// Carregar imagem (pseudo-código)
Image img = loadImage("texture.png");

// Criar textura OpenGL
GLuint texID;
glGenTextures(1, &texID);
glBindTexture(GL_TEXTURE_2D, texID);

// Copiar dados de imagem
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0,
             GL_RGB, GL_UNSIGNED_BYTE, img.data);
```

Passo 2: Mapear Textura a Vértices

```
glBindTexture(GL_TEXTURE_2D, texID);
glBegin(GL_QUADS);
    // Importante: Coordenada de textura ANTES da posição
    glTexCoord2f(0, 0); glVertex3f(-1, -1, 0);
    glTexCoord2f(1, 0); glVertex3f( 1, -1, 0);
    glTexCoord2f(1, 1); glVertex3f( 1,  1, 0);
    glTexCoord2f(0, 1); glVertex3f(-1,  1, 0);
glEnd();
```

Ordem Crucial: glTexCoord2f ANTES de glVertex3f

Passo 3: Definir Parâmetros de Textura

```
// Modo de preenchimento (repetição)
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);

// Filtros (ver secção abaixo)
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

Modos de Preenchimento (Wrapping)

Modo	Resultado	Uso
GL_REPEAT	Repete padrão	Padrões infinitos
GL_CLAMP	Borda fixa última cor	Evitar repetições
GL_CLAMP_TO_EDGE	Estende borda	Transições suaves
GL_MIRROR_CLAMP	Espelha	Efeitos simétricos

Amostragem e Aliasing

Oversampling (Textura > Pixels)

- **Problema:** Textura maior que pixels finais
- **Resultado:** Múltiplos texels por pixel → aliasing
- **Padrão:** Moiré (padrões estranhos)
- **Solução:** Mipmapping

Undersampling (Textura < Pixels)

- **Problema:** Textura menor que pixels finais
- **Resultado:** Múltiplos pixels por texel → pixelização
- **Resultado:** Perda de detalhe
- **Solução:** Interpolação

Filtros de Textura

GL_NEAREST (Point Sampling)

- **Método:** Escolhe texel mais próximo
- **Velocidade:** Mais rápido
- **Qualidade:** Aspecto pixelizado
- **Uso:** Pixel art, estilo retro

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
```

GL_LINEAR (Bilinear Filtering)

- **Método:** Interpola 4 texels vizinhos
- **Velocidade:** Mais lento que NEAREST
- **Qualidade:** Suave, sem artefatos
- **Uso:** Maioria das aplicações modernas

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
```

GL_LINEAR_MIPMAP_LINEAR (Trilinear Filtering)

- **Método:** Interpola entre mipmaps e dentro deles
- **Velocidade:** Mais lento
- **Qualidade:** Melhor a distâncias variadas
- **Uso:** Objetos distantes, paisagens

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

Mipmapping

Definição: Níveis de mipmap = pirâmide de texturas progressivamente menores

Nível 0: 1024 × 1024
Nível 1: 512 × 512
Nível 2: 256 × 256
Nível 3: 128 × 128
...

Geração de Mipmaps

```
glGenerateMipmap(GL_TEXTURE_2D);
```

Benefícios

- **Cache locality:** Mipmaps menores em cache
- **Antialiasing:** Reduz aliasing para objetos distantes
- **Qualidade:** Preserva detalhe onde relevante
- **Performance:** Reduz bandwidth de memória

Transparência

Alpha Test (Transparência Binária)

```
glEnable(GL_ALPHA_TEST);  
glAlphaFunc(GL_GREATER, 0.5); // Pixel ou é opaco ou totalmente transparente
```

Características: - Teste binário: $\alpha \geq \text{threshold} \rightarrow$ desenhar, senão descartar - **Vantagem:** Simples, sem sorting - **Uso:** Texturas binárias (dentro/fora), árvores em paisagens - **Problema:** Arestas duras

Alpha Blending (Transparência Parcial)

```
glEnable(GL_BLEND);  
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

Fórmula:

$$C_{final} = C_{triangulo} \times \alpha + C_{fundo} \times (1 - \alpha)$$

Requisito Crítico: Sorting por profundidade (distante $\rightarrow$ próximo)

Problema: Se renderizar próximo primeiro, depois distante com alpha: - Distante “sobrescreve” próximo (incorreto) - **Solução:** Ordenar triângulos por profundidade

Problema de Cycles

- **A sobre B, B sobre C, C sobre A** $\rightarrow$ sem ordem correta
- **Solução:** Subdividir triângulos em ciclos ou usar Order-Independent Transparency (OIT)

Componentes de Cor de Material

Cor Difusa (GL_DIFFUSE)

- Cor base refletida uniformemente
- Afetada por iluminação difusa
- Exemplo: Cor geral do objeto

Cor Ambiente (GL_AMBIENT)

- Cor sob iluminação indireta
- Não afetada por posição de luz
- Exemplo: Sombra base

Cor Especular (GL_SPECULAR)

- Cor dos reflexos brilhantes
- Afetada por posição de observador
- Exemplo: Mancha brilhante em metal

Emissão (GL_EMISSION)

- Cor emitida pelo material
- Não dependente de iluminação
- Exemplo: Material que brilha (neon, lava)

Shininess (GL_SHININESS)

- Controla tamanho da mancha especular
- Intervalo: $[0, 128]$
- Valores altos = material muito brilhante

CULLING E PARTIÇÃO

Back-Face Culling

Ideia: Não desenhar faces viradas para trás (longe da câmara)

Detecção

Para cada triângulo: 1. Calcular normal do triângulo: $\vec{n} = (\vec{v}_1 - \vec{v}_0) \times (\vec{v}_2 - \vec{v}_0)$
2. Calcular vetor câmara: $\vec{c} = \text{posicao_câmara} - \vec{v}_0$ 3. Teste: $\vec{n} \cdot \vec{c} < 0 \rightarrow$ face virada para trás

Em OpenGL:


```
glEnable(GL_CULL_FACE);  
glCullFace(GL_BACK); // Não desenhar faces traseiras
```

Benefício

- Elimina ~50% dos triângulos
 - Melhoria significativa de performance
-

View Frustum Culling

Ideia: Não testar objetos claramente fora do View Frustum

View Frustum

- Pirâmide definida pela câmara
- **6 planos:** Perto, Longe, Cima, Baixo, Esquerda, Direita
- **Ângulo FOV:** Define inclinação dos planos

Teste de Ponto

Para cada ponto e cada plano:

$$d = \vec{n} \cdot (\vec{p} - \vec{p}_0) > 0 \text{ (dentro)}$$

Se ponto está dentro de todos os 6 planos → dentro do frustum

Teste de Bounding Box

Para cada plano do frustum: 1. **p-vertex:** Vértice de caixa mais próximo ao lado positivo do plano 2. **n-vertex:** Vértice mais longe 3. **Se n-vertex dentro do plano:** Caixa pode estar dentro 4. **Se p-vertex fora do plano:** Caixa está definitivamente fora

Para cada plano_frustum:

```
se (caixa.p_vertex fora do plano):  
    return "fora"
```

```
return "dentro ou intersecta"
```

Estruturas de Partição Espacial

Bounding Volume Hierarchy (BVH)

Ideia: Árvore de volumes envolventes (caixas)

Raiz: Caixa englobando tudo
Esquerda: Caixa metade esquerda
Malha 1
Malha 2
Direita: Caixa metade direita
Malha 3

Teste: 1. Testar caixa raiz 2. Se intersecta, testar filhos recursivamente 3. Se ponto está em caixa folha, fazer teste detalhado

Binary Space Partition (BSP)

Ideia: Dividir espaço recursivamente por planos

Espaço original: 1 plano divide em 2 subespaços
Cada subespaço: Outro plano divide recursivamente

Vantagens: - Extremamente eficiente para Fog of War, Shadow Volumes - Ordem implícita (back-to-front para transparência) - Usado em Quake, Half-Life

K-d Tree

Ideia: BSP mas com planos alinhados aos eixos

Nível 0: Plano vertical (X)
Metade X negativa
Nível 1: Plano horizontal (Y)
Metade X positiva
Nível 1: Plano horizontal (Y)

Vantagens: - Construção mais rápida que BSP genérico - Melhor cache locality - Bem adaptado a ray tracing

Quadtree (2D) / Octree (3D)

Ideia: Dividir espaço em 4 (2D) ou 8 (3D) partes iguais

Octree (3D):

Raiz: Cubo
8 filhos, cada um é cubo 1/8 do tamanho
Recursivo até limite de densidade

Aplicações: - Terrenos: Quadrees para LOD (Level of Detail) - Volumetria: Octrees para dados 3D densos - Partições SVO (Sparse Voxel Octrees)

Estratégias de Partição

Estratégia	Vantagem	Desvantagem
Top-Down	Simples implementar	Mais divisões do que necessário

Estratégia	Vantagem	Desvantagem
Bottom-Up	Menos divisões	Mais complexo
Binária	Balanceada	Pode ser desbalanceada
Iguais	Cache friendly	Menos adaptada a distribuições

FICHAS DE CONSOLIDAÇÃO

Ficha 1: Transformações Geométricas

Problema 1.1: Matriz de Translação

Dado: Ponto $P = (2, 3, 5)$ **Operação:** Transladar por $T = (1, -2, 3)$ **Solução:**

$$P' = P + T = (3, 1, 8)$$

Problema 1.2: Composição de Transformações

Dado: Rotacionar 45° em Z, depois transladar $(5, 0, 0)$ **Ordem em código:**

```
glTranslatef(5, 0, 0); // Escreve segundo (aplica primeiro)
glRotatef(45, 0, 0, 1); // Escreve primeiro (aplica segundo)
```

Porquê: Matrizes se multiplicam da direita ($T \times R \times v = T(R(v))$)

Problema 1.3: Rotação em Torno de Ponto Arbitrário

Objetivo: Rodar ponto P em torno de C **Processo:** 1. Transladar para origem: -C 2. Rodar 3. Transladar de volta: +C

Matriz composta: $M = T(C) \times R(\theta) \times T(-C)$

Ficha 2: Iluminação

Problema 2.1: Cálculo de Iluminação Blinn-Phong (Completo)

Dados: - Ponto $P = (1, 0, 1)$ - Normal $\vec{n} = (0, 1, 0)$ (para cima) - Luz em $L = (2, 2, 2)$ - $K_d = 0.8$, $K_a = 0.2$, $K_s = 0.9$, $n = 32$ - $I_l = 1.0$, $I_{ga} = 0.2$ - Câmara em $C = (1, 1, 1)$

Passo 1: Vetor para a Luz

$$\vec{l} = \frac{L - P}{|L - P|} = \frac{(1, 2, 1)}{\sqrt{1^2 + 2^2 + 1^2}} = \frac{(1, 2, 1)}{\sqrt{6}} \approx (0.408, 0.816, 0.408)$$

Passo 2: Componente Difusa

$$\vec{n} \cdot \vec{l} = (0, 1, 0) \cdot (0.408, 0.816, 0.408) = 0.816$$

$$I_d = K_d \times I_l \times \max(0, 0.816) = 0.8 \times 1.0 \times 0.816 = 0.653$$

Passo 3: Componente Ambiente

$$I_a = K_a \times I_{ga} = 0.2 \times 0.2 = 0.04$$

Passo 4: Componente Especular (Blinn-Phong)

Vetor da câmara para ponto:

$$\vec{v} = \frac{C - P}{|C - P|} = \frac{(0, 1, 0)}{1} = (0, 1, 0)$$

Meio-vetor:

$$\begin{aligned} \vec{h} &= \frac{\vec{l} + \vec{v}}{|\vec{l} + \vec{v}|} = \frac{(0.408, 0.816, 0.408) + (0, 1, 0)}{|(0.408, 1.816, 0.408)|} \\ &= \frac{(0.408, 1.816, 0.408)}{\sqrt{0.408^2 + 1.816^2 + 0.408^2}} = \frac{(0.408, 1.816, 0.408)}{1.886} \approx (0.216, 0.963, 0.216) \end{aligned}$$

Produto interno normal-meio-vetor:

$$\vec{n} \cdot \vec{h} = (0, 1, 0) \cdot (0.216, 0.963, 0.216) = 0.963$$

Componente especular:

$$I_s = K_s \times I_l \times \max(0, \vec{n} \cdot \vec{h})^n = 0.9 \times 1.0 \times 0.963^{32}$$

$$0.963^{32} \approx 0.14 \text{ (muito pequeno devido ao } n=32 \text{ alto)}$$

$$I_s \approx 0.9 \times 0.14 = 0.126$$

Passo 5: Total (sem atenuação)

$$I_{total} = I_a + I_d + I_s = 0.04 + 0.653 + 0.126 = 0.819$$

Nota Crucial: Se tivesse atenuação $f_{att} = 0.5$:

$$I_{total} = I_a + f_{att} \times (I_d + I_s) = 0.04 + 0.5 \times (0.653 + 0.126) = 0.429$$

Ficha 3: Curvas Bezier

Problema 3.1: Ponto em Curva Bezier Quadrática

Dados: Pontos de controlo $P = (0, 0)$, $P = (1, 2)$, $P = (2, 0)$ **Parâmetro:** $t = 0.5$

Fórmula:

$$\begin{aligned} B(0.5) &= (1 - 0.5)^2 \times P_0 + 2(1 - 0.5)(0.5) \times P_1 + 0.5^2 \times P_2 \\ &= 0.25(0, 0) + 0.5(1, 2) + 0.25(2, 0) \\ &= (0, 0) + (0.5, 1) + (0.5, 0) \\ &= (1, 1) \end{aligned}$$

Problema 3.2: Propriedade de Convex Hull

Garantia: Qualquer curva Bezier está dentro do polígono de controlo **Impli-**
cação: Máximos e mínimos da curva estão próximos dos vértices

Ficha 4: Texturas

Problema 4.1: Coordenadas de Textura

Dado: Quad $[(-1, -1, 0), (1, -1, 0), (1, 1, 0), (-1, 1, 0)]$ **Textura:** 256×256 pixels

Mapeamento:

```
glTexCoord2f(0, 0); glVertex3f(-1, -1, 0); // Canto inferior esquerdo
glTexCoord2f(1, 0); glVertex3f( 1, -1, 0); // Canto inferior direito
glTexCoord2f(1, 1); glVertex3f( 1,  1, 0); // Canto superior direito
glTexCoord2f(0, 1); glVertex3f(-1,  1, 0); // Canto superior esquerdo
```

Ficha 5: Culling

Problema 5.1: Back-Face Culling com Regra da Mão Direita

Dado: Triângulo com vértices $(0, 0, 0)$, $(1, 0, 0)$, $(1, 1, 0)$ **Câmara:** $(0.5, 0.5, 1)$

Passo 1: Vetores das Arestas

$$\vec{v}_1 = (1, 0, 0) - (0, 0, 0) = (1, 0, 0)$$

$$\vec{v}_2 = (1, 1, 0) - (0, 0, 0) = (1, 1, 0)$$

Passo 2: Produto Cruzado (Regra da Mão Direita)

$$\begin{aligned}\vec{n} = \vec{v}_1 \times \vec{v}_2 &= \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{vmatrix} \\ &= \vec{i}(0 \cdot 0 - 0 \cdot 1) - \vec{j}(1 \cdot 0 - 0 \cdot 1) + \vec{k}(1 \cdot 1 - 0 \cdot 1) \\ &= (0, 0, 1)\end{aligned}$$

Passo 3: Teste de Visibilidade Vetor de um vértice para câmara:

$$\vec{c} = C - P_0 = (0.5, 0.5, 1) - (0, 0, 0) = (0.5, 0.5, 1)$$

Produto interno:

$$\vec{n} \cdot \vec{c} = (0, 0, 1) \cdot (0.5, 0.5, 1) = 1 > 0$$

Resultado: $\vec{n} \cdot \vec{c} > 0 \rightarrow$ **Face Visível** (normal aponta para câmara) - A face será desenhada - Ordem de vértices CCW (Counter-Clockwise) define Face Front

Problema 5.2: View Frustum Culling

Caixa: Vértices de (-1,-1,-1) a (1,1,1) **Frustum:** 6 planos

Processo: 1. Para cada plano, calcular p-vertex e n-vertex 2. Se p-vertex fora de algum plano $\rightarrow$ caixa fora 3. Senão $\rightarrow$ caixa pode estar dentro ou intersectar

Fichas com Soluções

Ficha Curvas e Superfícies - Q1

Questão: “Qual é a principal vantagem do Algoritmo de De Casteljau?”

Resposta: Estabilidade numérica. Usa apenas interpolações lineares sucessivas, evitando elevações a potências que podem causar erros de arredondamento. É particularmente importante para curvas de alto grau.

Ficha Texturas - Q2

Questão: “O que é Mipmapping e quando é usado?”

Resposta: Mipmapping é um nível de detalhe para texturas. Cria progressivamente texturas menores (1/2, 1/4, etc). É usado quando: - Texturas estão distantes (pixelização) - Performance é crítica (mipmaps menores em cache) - Aliasing é visível (moiré patterns)

O hardware escolhe automaticamente o mipmap apropriado baseado na distância.

Ficha Culling - Q3

Questão: “Porquê View Frustum Culling é mais eficiente que testar cada triângulo?”

Resposta: Porque testa grupos inteiros de triângulos simultaneamente (bounding volumes). Um teste de caixa pode descartar milhares de triângulos. Estruturas como BVH organizam recursivamente, permitindo poda exponencial do espaço de busca.

GRÁFICOS FUNDAMENTAIS PARA MEMORIZAR

A. Fluxo de Transparência no Espaço (Árvore BSP)

Problema: Alpha Blending requer ordenação correta (distante → próximo)

Solução: Árvore BSP divide automaticamente o espaço

ESPAÇO TRIDIMENSIONAL

Lado 1 (Atrás)

(Desenhar PRIMEIRO)

PLANO BSP

(Desenhar SEGUNDO)

Lado 2 (Frente)

Câmara está no Lado 2

Algoritmo de Travessia:

função DesenharBSP(nó):

 se nó é nulo:

 return

 lado_câmara = qual lado do plano a câmara está?

 lado_oposto = outro lado

 // Desenhar subárvore atrás

 DesenharBSP(nó.filhos[lado_oposto])

 // Desenhar polígono do plano

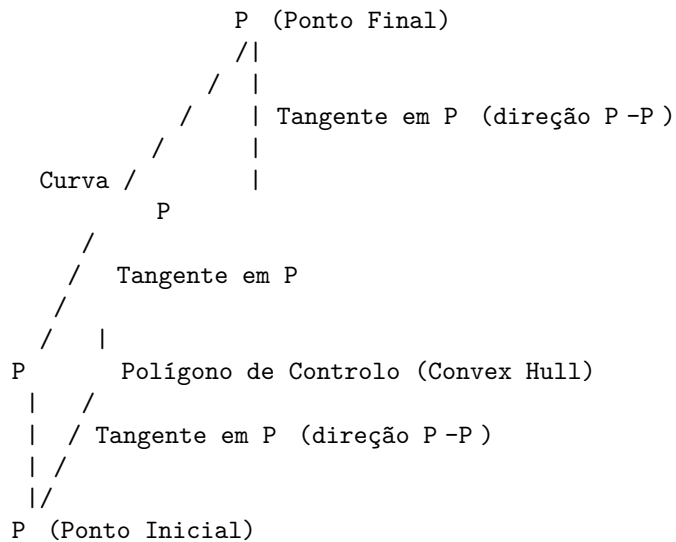
 DesenharPolígono(nó.polígono)

```
// Desenhar subárvore à frente
DesenharBSP(nó.filhos[lado_câmara])
```

Resultado: Ordem correta automaticamente garantida (back-to-front)

B. Curvas de Bézier Cúbicas e Convex Hull

Visualização dos 4 Pontos de Controlo:



Propriedades Críticas: 1. **Passa pelos extremos:** $B(0) = P$, $B(1) = P$ 2. **Tangente em P:** Direção do segmento $P \rightarrow P$ 3. **Tangente em P:** Direção do segmento $P \rightarrow P$ 4. **Envelope Convexo:** Toda a curva está dentro do quadrilátero $P P P P$

Para Continuidade C^1 (Suave Entre Curvas):

Curva 1: P, P, P, P
 $(P - P \text{ é a tangente em } P)$
 $|$
 $V \text{ (Devem ser colineares)}$
 Curva 2: Q, Q, Q, Q
 $(Q - Q \text{ é a tangente em } Q)$

Condição: $P = Q$ (mesmo ponto) E P, P, Q colineares (mesma tangente)

C. Mapa Conceptual de Estruturas de Partição Espacial

Quando Usar Cada Uma:

ESCOLHER ESTRUTURA DE PARTIÇÃO

V	V
BVH	Culling View Frustum
(Geral)	(Câmara)
	View Frustum Culling
	(6 planos do Frustum)
	Early discard de objetos
	fora da câmara

BSP Tree (Espaço inteiro)
Pros: Ordem back-to-front automática
Cons: Mais complexo, polígono pode dividir-se

K-d Tree (Eixos alinhados)
Pros: Construção rápida, ray tracing
Cons: Menos flexível que BSP

Octree (Divisão 8x recursiva)
Pros: LOD, volumetria
Cons: Cache miss em grandes dimensões

Quadtree (2D, terrenos)
Pros: LOD para terrenos
Cons: Apenas 2D

Hierarquia de Testes (Mais Rápido ao Mais Lento):

1. View Frustum Culling (6 planos)
↓ (Se passou, fazer teste mais detalhado)
 2. BVH / Octree (recursivo por distância)
↓ (Se passou, testar bounding boxes menores)
 3. Mesh Culling (Back-face culling)
↓ (Se passou, testar cada triângulo)
 4. Per-Pixel Culling (Alpha test)
-

RESUMO DE FÓRMULAS CRÍTICAS

Iluminação

- **Difusa:** $I_d = K_d \times I_l \times \max(0, \vec{n} \cdot \vec{l})$
- **Especular Phong:** $I_s = K_s \times I_l \times \max(0, \cos \alpha)^n$
- **Atenuação:** $f_{att} = \min(1/(c_1 + c_2d + c_3d^2), 1.0)$

Transformações

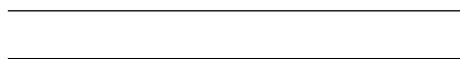
- **Translação:** $P' = P + T$
- **Escala:** $P' = P \times S$
- **Rotação:** Matrizes de rotação (consultar acima)

Curvas

- **Bezier Cúbica:** $B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3$
- **Hermite:** $B(t) = H_0(t)P_0 + H_1(t)T_0 + H_2(t)P_1 + H_3(t)T_1$

Espaços

- **Coordenadas Esféricas:** $x = r \sin \phi \cos \theta$, $y = r \cos \phi$, $z = r \sin \phi \sin \theta$



ARMADILHAS COMUNS NO TESTE

1. Ordem de Transformações

Erro Típico: Aplicar `glTranslate` DEPOIS de `glRotate` quando deveria ser ao contrário

Regra de Ouro: - Código escreve de baixo para cima - Mas matrizes aplicam-se de cima para baixo - Se quero rodar e depois transladar: `glTranslate` depois `glRotate` no código

```
// ERRADO (se queria rodar depois transladar)
glRotatef(45, 0, 0, 1);
glTranslatef(5, 0, 0);
```

```
// CORRETO (roda depois translada)
glTranslatef(5, 0, 0); // Escreve segundo
glRotatef(45, 0, 0, 1); // Escreve primeiro (aplica-se primeiro)
```



2. Coordenadas de Textura vs Posição de Vértice

Erro Típico: glVertex3f ANTES de glTexCoord2f

```
// ERRADO
glVertex3f(-1, -1, 0);
glTexCoord2f(0, 0);
```

```
// CORRETO
glTexCoord2f(0, 0);
glVertex3f(-1, -1, 0);
```

Porquê: Atributos são “colados” ao próximo vértice invocado. Se definir textura depois do vértice, o vértice fica com coordenadas antigas.

3. Normalização de Vetores em Iluminação

Erro Típico: Usar vetores não normalizados na fórmula de iluminação

$$\vec{n} \cdot \vec{l} = \frac{n_x \cdot l_x + n_y \cdot l_y + n_z \cdot l_z}{|\vec{n}| \times |\vec{l}|}$$

Se já normalizado ($|\vec{n}| = |\vec{l}| = 1$):

$$\vec{n} \cdot \vec{l} = n_x \cdot l_x + n_y \cdot l_y + n_z \cdot l_z$$

Sempre normalizar antes de usar em fórmulas!

4. Back-Face Culling e Ordem de Vértices

Erro Típico: Esquecer que a ordem importa para determinar “frente”

- **CCW (Counter-Clockwise):** Frente da face
- **CW (Clockwise):** Trás da face

Se o professor desenhar um triângulo e pedir “cullarás esta face?”, contar a ordem dos vértices!

5. Blinn-Phong vs Phong

Erro Típico: Confundir qual usar

Aspecto	Phong	Blinn-Phong
Fórmula	$\max(0, \cos \alpha)^n$	$\max(0, \vec{n} \cdot \vec{h})^n$
Intermediário	Vetor refletido	Meio-vetor
Velocidade	Mais lento	Mais rápido
Onde ver	Livros clássicos	Slides UMinho (provável)

Se o teste disser “calcula especular”, **assume Blinn-Phong** (mais moderno).

6. Alpha Blending Requer Sorting

Erro Típico: Desenhar triângulos transparentes em qualquer ordem

Resultado: Artefatos gráficos visíveis

Solução: 1. Sortear triângulos por profundidade (z do centróide) 2. Desenhar distantes primeiro, próximos depois 3. Usar fórmula: $C_{final} = C_{tri} \times \alpha + C_{fundo} \times (1 - \alpha)$

7. Mipmaps e Filtros

Erro Típico: Configurar GL_NEAREST para texturas distantes

// ERRADO para renderização normal

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

// CORRETO para maioria de casos

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
```

Quando GL_NEAREST: - Pixel art - Retro style - Performance crítica

8. Convenção de Coordenadas Esféricas

Erro Típico: Usar ϕ clássico quando devia usar α da UMinho

Se o teste falar “ângulo vertical a partir do plano XZ”: - É α (alpha) - $Y = r \times \sin(\alpha)$ — Seno no Y!

Se falar “ângulo polar a partir do eixo Y”: - É ϕ (phi) clássica - $Y = r \times \cos(\phi)$ — Cosseno no Y!

CHECKLIST FINAL ANTES DO TESTE

- ☐ **Iluminação:** Memorizar K_d , K_a , K_s , K_e e quando cada uma se aplica
 - ☐ **Transformações:** Praticar ordem de $glTranslate/glRotate/glScale$
 - ☐ **Curvas:** Entender que De Casteljau é estável numericamente
 - ☐ **Texturas:** Ordem: $texCoord$ ANTES de vertex
 - ☐ **Culling:** Testar Back-Face com produto cruzado + vetor câmara
 - ☐ **Transparência:** Se há alpha, SEMPRE ordenar por profundidade
 - ☐ **Espaços:** Conhecer os 4 espaços de coordenadas (Local, Global, Câmara, Ecrã)
 - ☐ **Funções OpenGL:** $glMaterialfv$, $glTexCoord2f$, $glCullFace$, $gluLookAt$
 - ☐ **Coordenadas Esféricas:** Verificar a convenção no enunciado (ou)
 - ☐ **Normais:** Sempre normalizar vetores na iluminação
-

Dicas Finais para o Teste

Lê com atenção: Se disser “elevação”, é seno; se disser “polar”, é cosseno
Desenha diagramas: Back-Face Culling, Bézier, BSP Tree ajudam a visualizar
Verifica normalização: Vetores em iluminação devem ter magnitude 1
Ordem é crítica: Transformações, atributos de vértice, sortagem de polígonos
Estima valores: Se pedir cálculo com números reais, arredonda sensatamente
Explica passo-a-passo: Mostre como calculou (professores apreciam)

Tens tudo aqui para dominares o teste!

Este resumo inclui: - Todas as fórmulas (iluminação, transformações, curvas)
- Todas as variáveis explicadas - Esquemas visuais (BSP, Bézier, Partition Trees)
- Coordenadas (Local, Global, Câmara, Ecrã, Esféricas) - Funções OpenGL concretas
- Fichas com soluções detalhadas - Armadilhas comuns evitadas

Boa sorte amanhã!